

Composable Live Programming with Engraft

ANONYMOUS AUTHOR(S)

Live, interactive tools can support a diversity of domain-specific programming tasks, from visualization authoring to data wrangling. Real-world programming, however, requires performing multiple tasks in concert, calling for the use of multiple tools alongside conventional code. Programmers lack environments capable of composing live tools to support these situations. To enable live composability, we contribute Engraft, an API for the Web platform that allows live tools to be embedded within larger live-programming environments like computational notebooks. Engraft enables several new forms of composition: not only embedding live tools inside live environments, but also embedding live environments within each other and embedding live environments in the outside world, including conventional codebases. We demonstrate Engraft with examples from diverse domains, including web-application development and physics education. By providing composability, Engraft can help cultivate a cycle of use and innovation in live programming.

CCS Concepts: • **Human-centered computing** → **Graphical user interfaces**; • **Software and its engineering** → **Integrated and visual development environments**; **Application specific development environments**.

Additional Key Words and Phrases: live programming, visual programming, end-user programming, composition, computational notebooks, GUIs

ACM Reference Format:

Anonymous Author(s). 2018. Composable Live Programming with Engraft. In . ACM, New York, NY, USA, 30 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 INTRODUCTION

People use a vast range of rich, specialized computer interfaces in daily life. We use spreadsheets to track accounts, graphics editors to create images, and timeline views for video editing. These interfaces are crafted around their domains, offering representations that afford direct manipulation and meaningful feedback.

Turning our gaze to programming, this richness disappears. Conventionally, programming takes place in a uniform world of textual code. Parsing text, processing images, building a visualization, and chaining steps of a process are all distinct tasks, but a programmer’s screen looks the same no matter which they are working on. “Domain-specific” languages can offer separate dialects for these separate situations, but even with DSLs, programming still takes on a textual form, deprived of the direct manipulation and live feedback that characterize everyday computer use.

Alternatives to textual code have been explored since the advent of interactive computing. Some of the earliest user interfaces developed were visual programming languages, using formalisms like flow charts to define programs [9, 10]. In this paper, we build on work in *live programming* [42]: “programming tools which provide immediate feedback on the dynamic behavior of a program even while programming” [35]. Live programming aims to provide the immediate feedback that we expect from rich computer applications. Live programming might be most helpful via a rich variety of domain-specific tools, with particular tools crafted to help programmers accomplish particular tasks. For example:

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 Association for Computing Machinery.

Manuscript submitted to ACM

- Gneiss [4] is a tool for making interactive, data-driven web applications through direct manipulation.
- Lyra [37], Data Illustrator [22], and Charticulator [36] are visualization authoring systems that let authors create expressive data visualizations without using traditional code.
- Wrangler [17] is a programming-by-demonstration interface for data wrangling. With it, users can define complex scripts to clean up and reshape tabular data by directly manipulating a table, rather than writing lines of code.

These tools use novel techniques to make programming tasks concrete, visible, and approachable. However, while particular tools may find their niches, the role of interactive tools in programming *at large* remains marginal. Programmers may go their entire careers without leaving the world of static text. Furthermore, the potential of live tools to extend programming power from trained software engineers to end-users in diverse disciplines remains unrealized. What fundamental limitations do live-programming systems have today which, if addressed, might bring them into broad use?

We contend that a main factor limiting real use of live programming tools is their present lack of composability. Conventional programming derives its power in large part from composability. This composability is so fundamental to programming that we take it for granted. We assume that *of course* we can combine multiple Python libraries when we write our scripts, and nest our *if* statements and *for* loops however we like. The ability to freely combine building blocks is what makes programming a medium with infinite expressive potential.

This composability does not exist in the world of rich user interfaces, including live-programming tools. As an example, suppose a digital artist wants to make a website that displays summaries of random Wikipedia articles.¹ This project involves multiple steps:

- (1) Call the Wikipedia API to get raw data about articles.
- (2) Process a JSON response to extract out relevant information.
- (3) Format this information into an attractive web page.
- (4) Host the above process on a publicly available site.

This workflow is heterogeneous. We can imagine separate live-programming tools for each of these steps, like a tool which displays a JSON structure and lets a user pick the information they want (for step 2), and a tool which lets a user build a data-backed web page through direct manipulation (for step 3). Could the artist use such tools together to accomplish their larger goal? Right now, they can not. Ad-hoc approaches are possible, including tools that generate code or hard-wired tool assemblies for specific workflows, but none of these can replicate the free-form composability of conventional code without losing liveness and immediacy.

We envision general-purpose programming systems combining three essential elements. Many systems combine two of these elements, but few research systems—and none in common use—include all three:

- **Liveness:** Feedback about the program’s dynamic behavior deeply permeates the programming experience.
- **Domain-specific richness:** Programmers use interactive representations designed for the domains they work with.
- **Composability:** Tools can be freely combined to support diverse workflows and accomplish larger goals.

We present *Engraft*,² a system for live composition of live, domain-specific tools and environments. Engraft takes the form of an API on the Web platform. An Engraft-compatible tool follows a standard interface for embedding, allowing it

¹This use-case is inspired by Pipkin [32], a live-tweeted coding adventure.

²Grafting is “a horticultural technique whereby tissues of plants are joined so as to continue their growth together” [47]. This provides an apt analogy for the way Engraft allows trees of heterogeneous live tools to work together.

to be embedded together with other tools within live environments like notebooks. In this way, disparate task-specific tools can be joined together to solve real problems, while maintaining liveness among them.

A number of key design decisions shape the Engraff API. Engraff tools are given *freedom of implementation*. A tool must implement a common interface, but its internal implementation is otherwise unconstrained. Engraff tools *can be embedded anywhere in the Web platform*, including embeddings inside *other tools*, as well as in diverse applications and development contexts. Engraff tools communicate with their hosts using *streams of pure values*. A tool is given values by its host and it provides values in return. Engraff tools expose *computational and UI behaviors, with computational behavior primary*. An Engraff tool can run without rendering any UI, but not visa versa. Finally, Engraff tools *are not required to generate or edit underlying textual code*. Engraff explores a vision of live programming untethered from a textual-code source of truth.

By following these decisions, Engraff enables **three forms of composition**, all of which are underexplored and two of which are novel:



Live tools inside live environments. The goal already described: joining task-specific tools together in a shared live environment.



Live environments inside live environments. Environments can embed in other live environments, with nested scopes at multiple levels in the programming process.



Live tools inside the outside world. Engraff tools (including environments) can be embedded into applications or codebases.

1.1 Summary of Contributions

- We identify *composability* as an essential element missing from the current live-programming ecosystem.
- We articulate three forms of composition that are necessary for live programming to match conventional programming in expressiveness and practical utility.
- We present a model for an API with which heterogeneous live tools can be composed together via the three forms of composition we articulated.
- We present our prototype of this API, called Engraff, together with an assortment of example tools that showcase interaction styles that might be used in tools in the future.
- We show, through examples, how our prototype makes a variety of programming activities visible and interactive.

2 RELATED WORK

Engraff combines three essential elements: *liveness*, *domain-specific richness*, and *composability*. Each of these elements have long histories in the literature, though it is rare for all three to be combined at once (Figure 1). We first look at pairwise combinations of these elements, then discuss the special triple intersection consisting of projects most similar to Engraff.

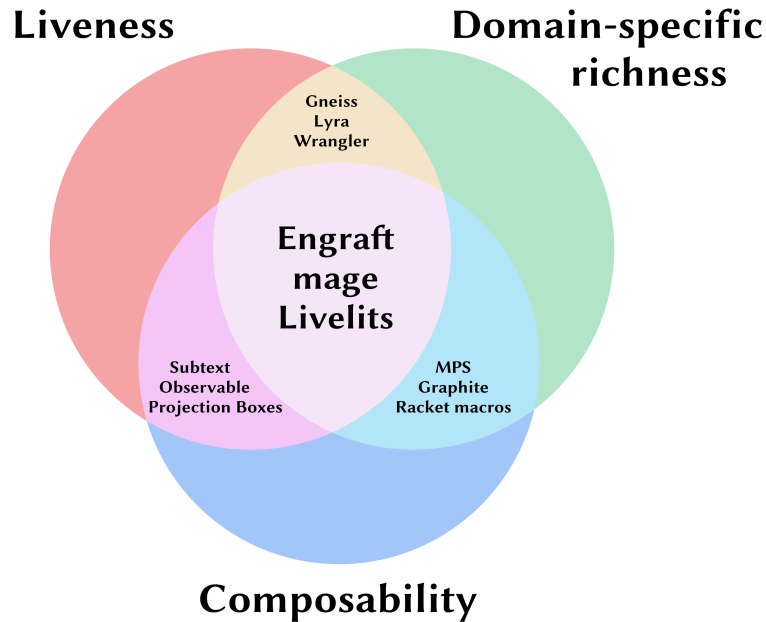


Fig. 1. Pairwise and three-way intersections between liveness, domain-specific richness, and composability.

2.1 Liveness + Domain-specific richness

Engraff is inspired by domain-specific programming tools like Gneiss [4], Lyra [37], and Wrangler [17]. These tools replace complex, opaque programming processes with direct interactions. To achieve this, they leverage novel programming paradigms (such as Gneiss’s extensions of the spreadsheet model) and interaction techniques (such as Wrangler’s use of “programming by demonstration”). We see enormous potential in tools like these.

However, these tools are fundamentally isolated. The challenge of combining them, or integrating them into conventional programming systems, undermines the benefits of their liveness, and arguably limits their use. Lacking such composability, developers of novel interactive programming tools have taken a variety of ad-hoc approaches to integrate them into larger workflows.

One approach is simply “copy-paste”. For example, regex101, Debuggex, and RegExr offer live debuggers, visualizers, and documentation to help programmers write regular expressions. These tools are not integrated with developer tools like IDEs. One simply copies regular expressions in and out as needed. Though quite a bit more complex than a regular-expression helper, Wrangler [17] similarly “integrates” with larger programming tasks via “copy-paste” of generated code. Although this works, consider what is lost.

There is a loss of *liveness*. In a copy-paste system, the user must provide sample input by hand. This tedious extra step is a place where mistakes can be propagated if the user’s assumptions about what the input data looks like do not match reality. There is also a loss of *persistence*. With one of these online tools, a programmer may curate a set of test inputs, but this ensemble is not persisted when the regular expression is copy-pasted back to code. Finally, there is a

loss of *collocation*. Using an online tool requires leaving the application code and entering a new space. With this comes a potential for disorientation and distraction.

Academic researchers developing novel interactive programming tools have generally not focused attention on how they might be integrated into larger programming workflows, preferring to focus on the tools themselves. Furthermore, even if tool-makers go beyond the call of duty and build tight integrations into larger environments, these integrations will be *ad-hoc*. They must be built from scratch, as was done to integrate Voyager [48] into JupyterLab. Generally, integrations must be built separately for each development environment. In contrast, Engraff provides a single API that tools and hosts can use for live, fully persistent, colocated embedding.

2.2 Composability + Liveness

Next, we consider systems combining composability and liveness,³ albeit without domain-specific richness. Most live programming editors fall into this space. We place these into three categories:

Liveness within textual code. One common recipe is to augment textual-code editors with in-context displays showing run-time behavior. Rauch et al.'s "Babylonian-style Programming" [34] surveys the state of the art in this category as of 2012. They evaluate eight existing editors [6, 13, 15, 18, 44–46], before adding their own to the list. Notable entrants to this category since 2012 include Swift Playgrounds in Xcode [2] and Projection Boxes [21]. These editors imbue conventional textual code with powerful new forms of visibility. As they restrict their attention to this conventional textual code, they do not provide opportunities for rich task-specific representations and interactions.

Liveness between textual code cells. Rather than threading live feedback into traditional codebases, other editors provide special interfaces where code is broken up into "cells". These editors can then provide visibility into values flowing between cells. We call such editors "live environments", as they provide liveness to environments where pieces of code are composed. The paradigmatic live environment is the spreadsheet. More recent live environments include Observable [27] and Natto [40], which both augment JavaScript programming in-the-small with a live structure in-the-large: Observable via a computational notebook and Natto via nodes and wires on a canvas. While environments like these play an important role in building programs with Engraff, they do not, by themselves, enable natural domain-specific interactivity. Specifically: (1) Cells can render UI, but this UI cannot persistently modify program state.⁴ (2) They do not support hierarchical nesting of tools. (3) They are both monolithic web applications which cannot themselves be embedded into other live environments or outside applications.⁵

Liveness with structured editors. Finally, we have *structured*, or *projectional*, editors which are not based on textual code or cells thereof, but which instead allow interactions directly with structured representations of code. Many structured-editor projects involve live feedback, including Enso [11], Lamdu [23], PANE [14], and Subtext [7]. To our knowledge, no live structured editor projects have added custom domain-specific components to their structures, though it would seem to be a natural direction.

³Some systems are live only in that they display the output of a program, and try to update this display in response to changes in the program. Examples include "live reloading" and "hot reloading" systems popular for application development. These are at the coarser end of a scale measuring the *granularity* of liveness. At the other end of the scale are systems that exhibit fine-grained liveness: making visible the dynamic behavior of smaller programming constructs like intermediate values and control flow.

⁴Observable and Natto have both implemented particular, ad-hoc tools that can persist state: Observable's Data Table Cell [28] and Natto's Template Panes. These extensions are built into their respective platforms and can access capabilities that are not available to user-authored tools.

⁵Observable notebooks and Natto canvases can be invoked via JavaScript module export, but someone using this export cannot see their own inputs flowing through the notebook/canvas, nor do they have the ability to modify the notebook/canvas's behavior in context.

2.3 Composability + Domain-specific richness

The final pairing integrates domain-specific representations and interactions into general-purpose programming environments, albeit static ones. A number of projects have explored this possibility. Andersen et al. [1] describes a visual macro system for Racket which lets users define interactively editable visual syntaxes embedded inside of arbitrary Racket code. Graphite [31] extends a Java IDE with pop-up “palettes” to bidirectionally edit literals in code. MPS (Meta Programming System) [16] is a workbench for developing projectional domain-specific languages. MPS languages typically look mostly like conventional code, but they can embed custom interfaces like tables and state-machine diagrams.

These approaches differ from Engraft along many axes, but by far the most salient is their lack of liveness. In all the above cases, visual representations embedded in code represent static syntax, and do not show information about dynamic program behavior. All of the tools in Engraft benefit from liveness, and some, like Extractor and Formatter, derive their entire power from allowing the user to act directly on data in the system. These sort of tools are not possible in a system without liveness.

2.4 Combining All Three

Engraft joins two recent projects that combine liveness, domain-specific richness, and composability: mage [19] (“an API for allowing smooth transitions between GUI and code work in notebooks”) and Hazel’s livelits [29] (“user-defined GUIs embedded persistently into code”). These projects share both motivations and common design elements with Engraft. However, Engraft goes beyond them by enabling new forms of composition.

While mage and livelits support joining tools together in a shared live environment, Engraft further supports embedding environments within environments, and embedding live tools within the outside world. Because its API is tied to its Jupyter host, mage tools cannot be embedded within each other, but must follow the flat structure of a Jupyter Notebook.

Unlike mage tools, livelits are nestable within each other and other Hazel [30] constructs, but a smaller variety of tools are possible than with mage or Engraft. Live environments like notebooks can not serve as livelits, as livelits’ typing discipline is not expressive enough to represent these dynamic environments.⁶ Livelits, as the name “live literals” suggests, have so far been restricted to widgets like sliders and color pickers. Engraft supports a broader range of tools, from nestable environments like Notebook to programming-by-demonstration tools like Extractor and Formatter.

Engraft also differs from livelits in our implementation strategy. While livelits are built in Hazel, a self-contained research platform, Engraft is built in the Web platform, where it can grow through contact with actual use.

3 LIVE COMPOSABILITY WITH ENGRAFT

As we previewed in the introduction, Engraft enables three forms of live composition: **live tools inside live environments**, **live environments inside live environments**, and **live tools inside the outside world**. Our initial work on Engraft was driven by the first of these. The architecture that we designed in response to this goal proved to be more versatile than we expected, enabling the last two as an emergent byproduct. In retrospect, we believe all three of these forms will be important to make programming with live tools a viable alternative to static code.

This section describes the end-user experience of building programs with Engraft, making use of these three forms of composition. Later, in §4 and §5, we discuss the design and implementation of the underlying Engraft API.

⁶The Hazel environment itself can be embedded in livelits, but this still means that developers do not have the benefit of experimenting with alternative live environments.

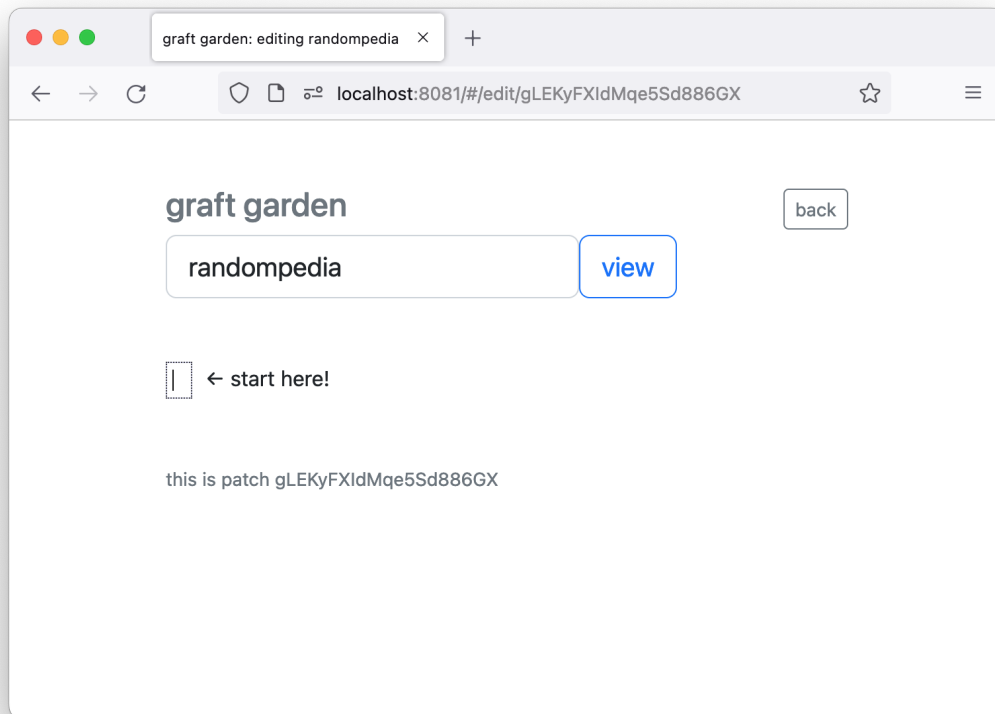


Fig. 2. A new “patch” (project) on Graft Garden.

3.1 Live tools inside live environments

Earlier, we presented an example of a digital artist who wants to make a website that displays summaries of random Wikipedia articles. Let’s examine how this artist might accomplish their goal by composing live tools together in a live environment with Engraff.

They start by loading the Graft Garden website. This is a web application which provides a convenient starting point for using Engraff tools. Graft Garden will also host the output of these tools as a separate web page, as we will see later.

Once the artist tells Graft Garden to make a new project, they are presented with a blank page (Figure 2), containing a single small code-editor box, called a *slot*. This slot is the starting point for everything they will do.

The artist knows that their project will involve combining together multiple steps. So they would like to work in a *live environment* that allows multiple computational steps to be combined together in fluid and flexible ways. Example environments include computational notebooks (like Observable), free-form canvases of nodes and wires (like Natto), and spreadsheets (like Excel). Engraff is not built around any one of these environments in particular. As versions of all of these have been implemented as Engraff tools, the artist can take their pick of whichever suits their needs.

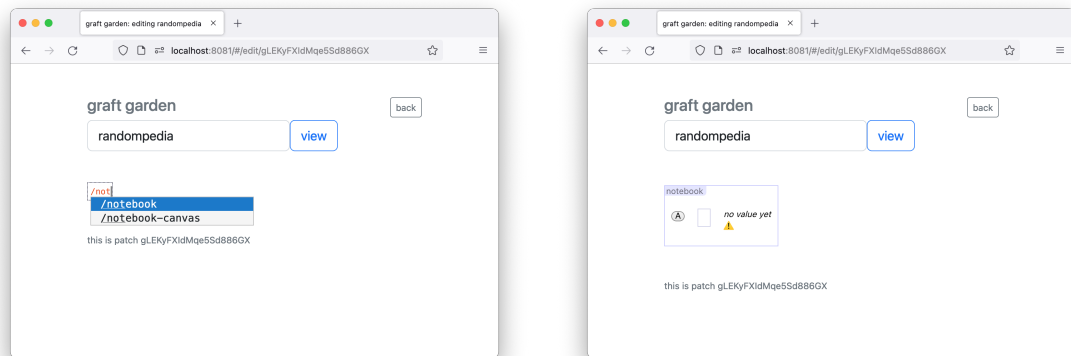


Fig. 3. The artist begins to type `/notebook` into the slot, and selects the autocomplete suggestion. A notebook is inserted into the slot.

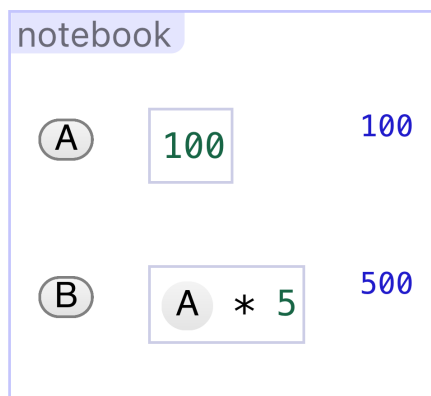


Fig. 4. Example usage of a notebook. The second cell (B) refers to the first (A).

In this case, the artist picks a computational notebook. They do this by beginning to type `/notebook` in the slot, and selecting ‘notebook’ from the autocomplete menu that pops up. Once they do this, the slot is replaced with the notebook interface (Figure 3).

This is a reactive notebook in which a user can type code snippets into cells. The notebook will evaluate these snippets and show their resulting values alongside the cells (Figure 4). Cells receive default names (like in a spreadsheet), but can be renamed. Cells can refer to one another, and the notebook ensures that a cell is re-evaluated when one of its references changes.

Let’s return to the artist. The first thing they need to do is get hold of data from the Wikipedia API. Rather than write networking code directly, they type `/request` into the first cell, and select ‘request’ from the autocomplete menu that pops up. This inserts a Request tool into the cell. The notebook’s cells are slots, just like the root slot the artist used to invoke the notebook.

The Request tool has two slots: one to provide a URL and one to provide an object of query parameters (Figure 5). As the artist skims the Wikipedia API documentation, they experiment with different query parameters. The tool resends the query with every change, so the artist can see the effects of their changes live.



Fig. 5. A Request tool inserted into the notebook. The artist has provided contents for its “url” and “params” slots. The request’s JSON response is shown to the right. Long string values are automatically placed in scrolling sub-windows.

Just like a snippet of code typed into a computational notebook, the Request tool produces *output*, which is returned to the notebook that hosts it. The notebook can then display this output live, and make it available to other cells by reference.

Examining the output, and playing with query parameters, the artist eventually finds a configuration they like. It delivers five random articles, complete with titles and HTML-summary “extracts”.

While the artist can see the data they want in the API response, the task of extracting it out of the complicated JSON structure is a bit intimidating. Fortunately, they know a second tool, called the Extractor tool⁷ (Figure 6). This tool accepts JSON data as input and presents an interface that allows the user to select data values by clicking on those values directly. The tool generalizes from these clicks, providing an output structure with the values the user wants. The result is equivalent to writing code that loops over arrays and objects with chains of property accesses.

This tool is more interesting, in its liveness, than the Request tool. The Request tool offered a convenient form interface for specifying a request, but that interface was structurally similar to working with code. In contrast, the Extractor tool’s interface relies on live input data to support programming-by-demonstration.

Now that the artist has extracted the data they care about, they want to reshape it into an attractive interface. For this they use a Formatter tool⁸ (Figure 7). Given a JavaScript data structure, Formatter automatically suggests a way to format the data into HTML output. It also provides direct-manipulation handles the user can use to choose different options for this formatting. Like Extractor, Formatter uses live data to power a programming-by-demonstration interface. The artist hands Formatter the output of Extractor, and does a bit of clean-up.

The artist has now built a live computational notebook, consisting of three live tools:

- (1) a Request tool, to get data from the Wikipedia API,

⁷Extractor is similar to an interaction found in Gneiss [4].

⁸Formatter is similar to a (second) interaction found in Gneiss [4], and is also inspired by Yoshiki Schmitz’s work [38].

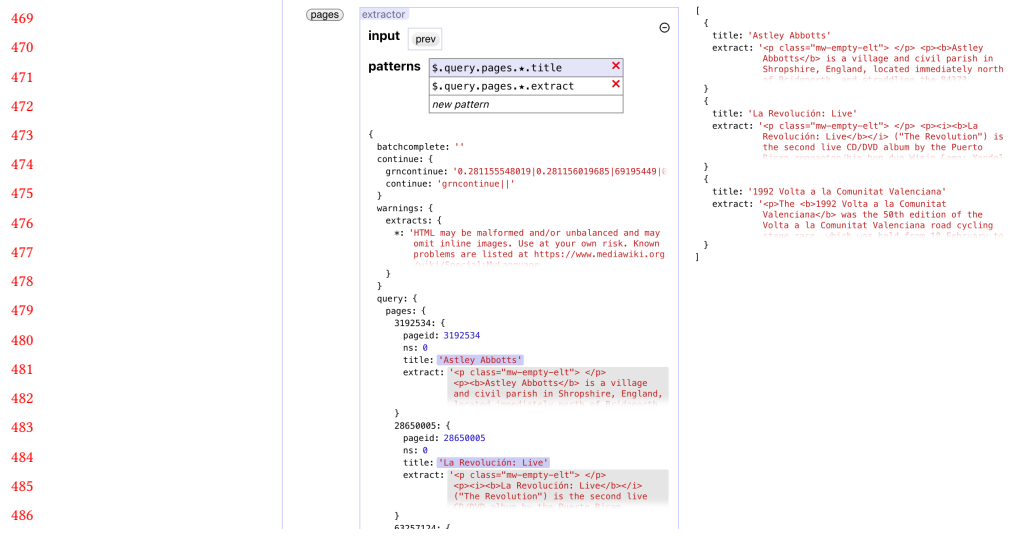


Fig. 6. An Extractor tool inserted into the notebook below the Request tool. It refers to the previous cell as input. The artist has selected various data fields by clicking on them. The tool's output is shown to the right.

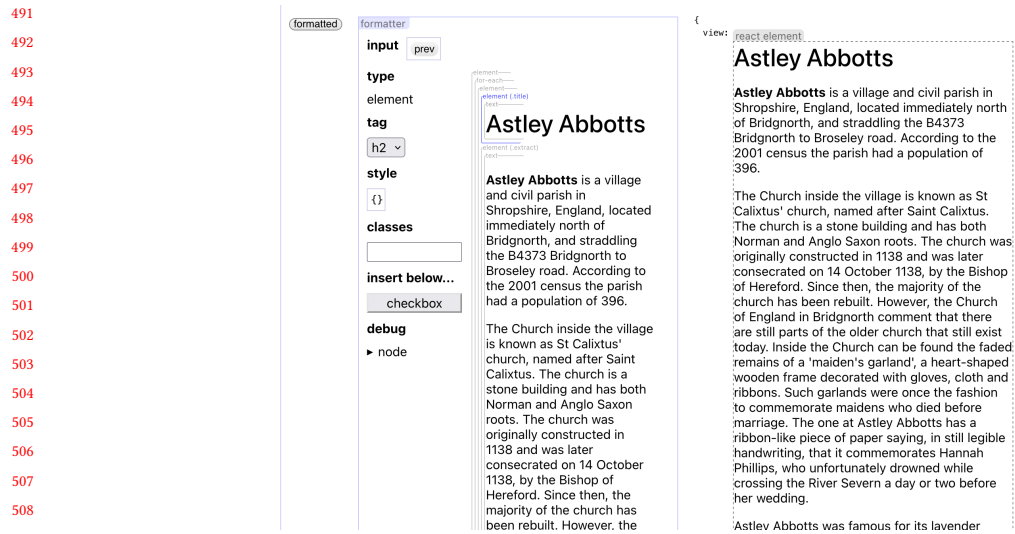


Fig. 7. A Formatter tool inserted into the notebook below the Extractor tool. It refers to the previous cell as input. The artist has used its interface (including an inspector on the left) to specify how the extracted data should be formatted. The tool output is shown to the right.

- (2) an Extractor tool, to pluck out the data they need in a clean format, and
- (3) a Formatter tool, to shape the data into an HTML document with the appearance they desire.

The last step is to “deploy” this living program to a website, where a visitor will receive their own random set of articles. Conveniently, Graft Garden, the web application that has hosted their work so far, has already done this. The

output of the last cell in the notebook (the Formatter, in this case) is returned back to Graft Garden. Graft Garden provides a shareable link that displays this final return value without visible tool UI (Figure 8). Note that the Wikipedia articles shown here are different than above, as the artist’s program runs anew every time this page is loaded.

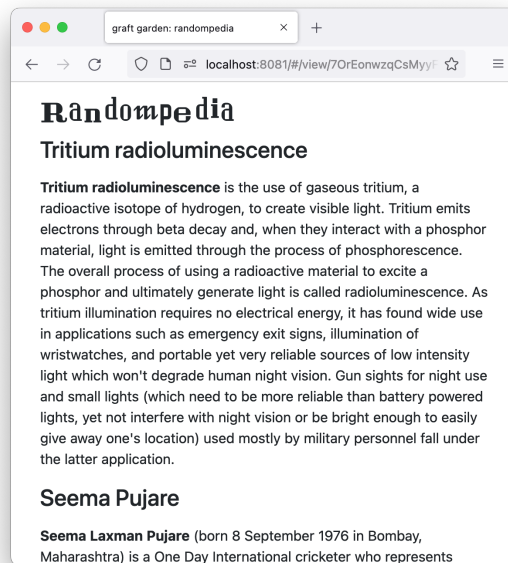


Fig. 8. The artist’s final creation: a dynamic website invisibly powered by their Engraff program. It is available at a separate URL.

Our artist has not had to type any code to create this dynamic web page, aside from experimenting with API query parameters. A “results not typical” disclaimer applies here: In most cases, more JavaScript coding will be required to accomplish real-world programming goals with Engraff. This fits with our vision, as Engraff is built to gradually extend conventional programming workflows. There should be nothing stopping a user from using traditional textual code whenever it is useful or necessary to do so. But when using Engraff, a new possibility opens up: replacing certain steps with interactive, direct-manipulation tools.

In the story above, the artist chose to compose their tools inside of a Notebook. As we mentioned earlier, this is not the only live environment an Engraff user may choose to use. For different tasks, different environments may be preferred.

Suppose a user wants to construct an arrow graphic to represent a vector of x - and y -values, perhaps as part of an interactive explanatory diagram. Figure 9 shows how they might do this in a Notebook. It then shows the same logical structure expressed in a new environment (loosely modeled on Natto [39]⁹), called a NotebookCanvas. Cells on a NotebookCanvas work the same way as on a Notebook, but they can be freely dragged around a canvas and resized however the user likes.

The user might choose to use NotebookCanvas over Notebook for a number of reasons: higher visual density, space to organize nonlinear data-flows, and a looseness that avoids some “negative effects of prematurely or unnecessarily

⁹A more ambitious adaptation of Natto, which could also be implemented with Engraff, would connect cells with wires rather than references.

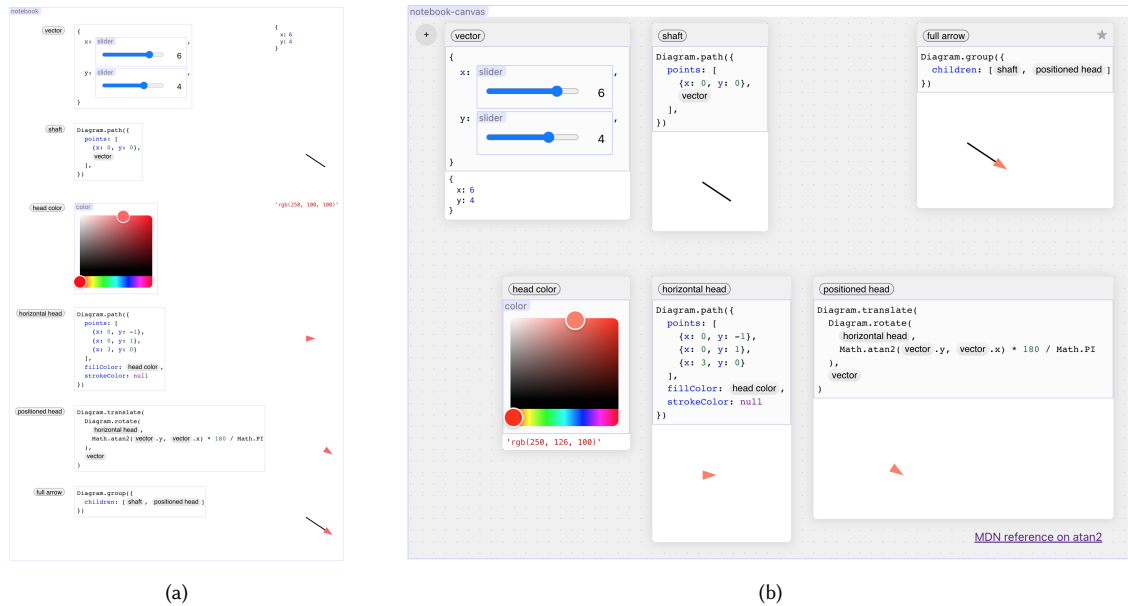


Fig. 9. (a) A Notebook in which a user builds an arrow graphic for a vector. (b) A NotebookCanvas consisting of the same cells, but arranged spatially.

imposing a structure” [41]. The user has this choice because Engraff decouples tools from environments, making it possible to choose the right environment for the job without losing access to the right tools.

3.2 Live environments inside live environments

Live environments are powerful building blocks for live programming. The above example uses a Notebook to link together three live tools. However we have found that one environment, by itself, is often not enough, because a single environment is “flat”.

To explain what we mean by *flat*, suppose an Observable user has an array of complex data elements. They would like to process each element of this array to obtain a new version of the array, a “map” operation. They are free, in Observable, to write a cell that performs this map operation:

```
newArray = oldArray.map((element) => {
  return step3(step2(step1(element)));
});
```

Here, the per-item processing function is complex, consisting of multiple steps (function calls). A selling point of a computational notebook is that it can split separate steps into separate cells, providing visibility into intermediate values. Here, Observable’s interface is not available *inside* this map operation, because the entire map operation needs to be inside a single cell. The benefits of liveness and visibility that Observable offers are lost in this deep structure. Observable is *flat*, and programming, in general, is not.

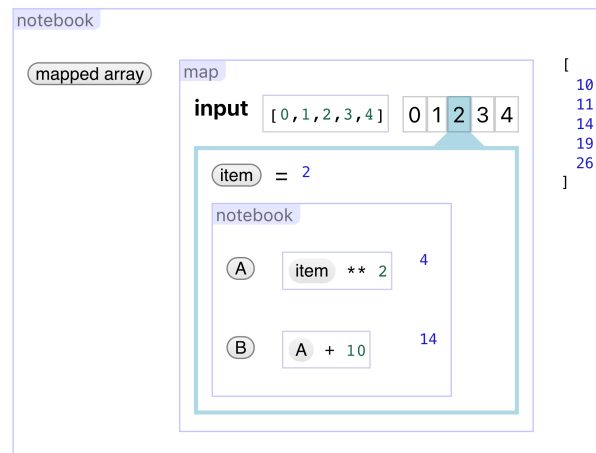


Fig. 10. A toy example, showing Notebook inside of Map inside of Notebook. Map has been given the array $[0, 1, 2, 3, 4]$ as input. Its inner notebook squares an item of an array and then adds 10, in two separate cells. The user has selected index 2 in the Map tool as the example they would like to display in the inner notebook.

In contrast, the Engraff architecture suggests that environments could themselves implement the API of an Engraff tool, becoming tools themselves. This means that live environments can be embedded inside of other live environments, allowing these environments to reflect a nested structure in the computation being performed.

For instance, we have built a tool called Map, designed to support a user mapping through an array. Map takes an input array and provides a slot for a “per-item tool”. Map shows the per-item tool being run on a single element of the array, so that the user has concrete data to inform their experience of live-programming the per-item tool. The user can use Map’s interface to select which element of the array they would like to use as their example, so they can test that their per-item tool performs well across variations. To compute its final output, Map also runs a copy of the per-item tool on each element of the input array, though most of these executions are invisible. (This is a good example of why it is important that tools can run without their interfaces being rendered.) Figure 10 shows a notebook embedded inside of Map, so that multiple stages in the per-item tool can be examined live as they are programmed.

To see Map used in a real-world context, let’s look at an “image quilt” generator made with Engraff (Figure 11). Starting with a user-supplied query, like “abstract” or “cat”, this web application displays a dense array of annotated artworks.

The Engraff program behind this application starts by querying the Art Institute of Chicago API for matching works of art. This returns an array of objects representing works of art. To make the quilt, we need to turn each element of this array into a composite of image and text. We can do this with Map (Figure 12).

By splitting the per-item process into multiple steps in the Notebook, we receive immediate feedback about each step. Is the URL of the image being generated correctly? How does the composite look with its current styling? (One can certainly imagine this composition step being replaced someday with a direct-manipulation tool!) Once the array of HTML elements is returned by Map, it can finally be composed into the quilt.

Programming is full of nested abstractions, so mapping an array is only one example of where it can be valuable to nest environments. As a very different example, consider programming a physics simulation. We are inspired here by the Bootstrap curriculum, which uses programming to teach algebra, physics, and computation to students in grades

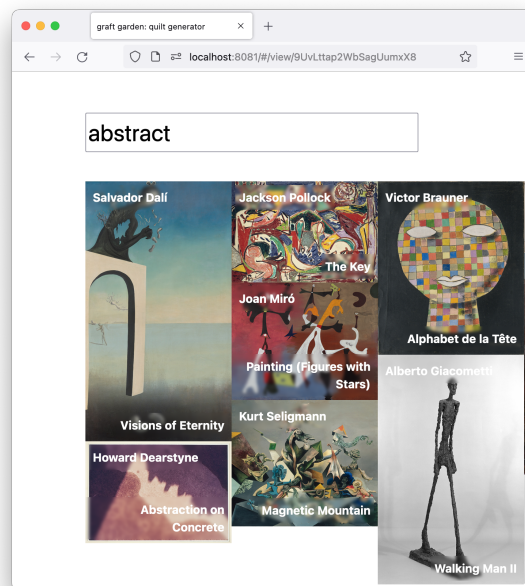


Fig. 11. The image quilt application, displayed in Graft Garden’s “view” mode. The images shown are a result of the user typing “abstract” into the search box.

5-12 [3]. We adopt a functional structure for our simulations similar to Bootstrap’s “reactor” [33], where a simulation is defined by:

- a *state* value, initialized to a certain value,
- a way to *view* the state, and
- a way to *update* the state on each time step.

A tool called Simulation lets a user define each of these pieces in slots. It can be useful to insert a live environment into one of these slots – say, to break down the “update” into steps. Here, a Notebook in the “update” slot of a Simulation describes a bouncing ball behavior (Figure 13).

By dragging the “step” slider, the user can see the “view” and “update” in the context of that particular step. Here, for instance, we see that the “x bounce” cell in the notebook has evaluated to `true`, so its `x` velocity will be inverted in the next step, as it bounces off the right-hand side of the box. Scrubbing through the time steps with the slider, the user can check that the pieces of their computation do what they expect, even as conditions change.

Simulation could have been implemented as a top-level application, rather than a tool. But implementing it as a tool provides even more benefits. In the example above, Simulation is in fact embedded in a larger Notebook (not shown). This larger notebook provides the shared variables “width” and “height” that the Simulation’s different slots can refer to. The Simulation tool also provides output of its own back to the Notebook: a trace of all the states the simulation passes through. This trace can be used, live, in other cells to analyze the output of the simulation. Here, we feed it into a tool that embeds Voyager 2 [48], a system for visual data exploration. Using Voyager 2’s interface, the user plots the `x` position over time (Figure 14). The bounce after time-step 8 is visible in this plot in a new way.

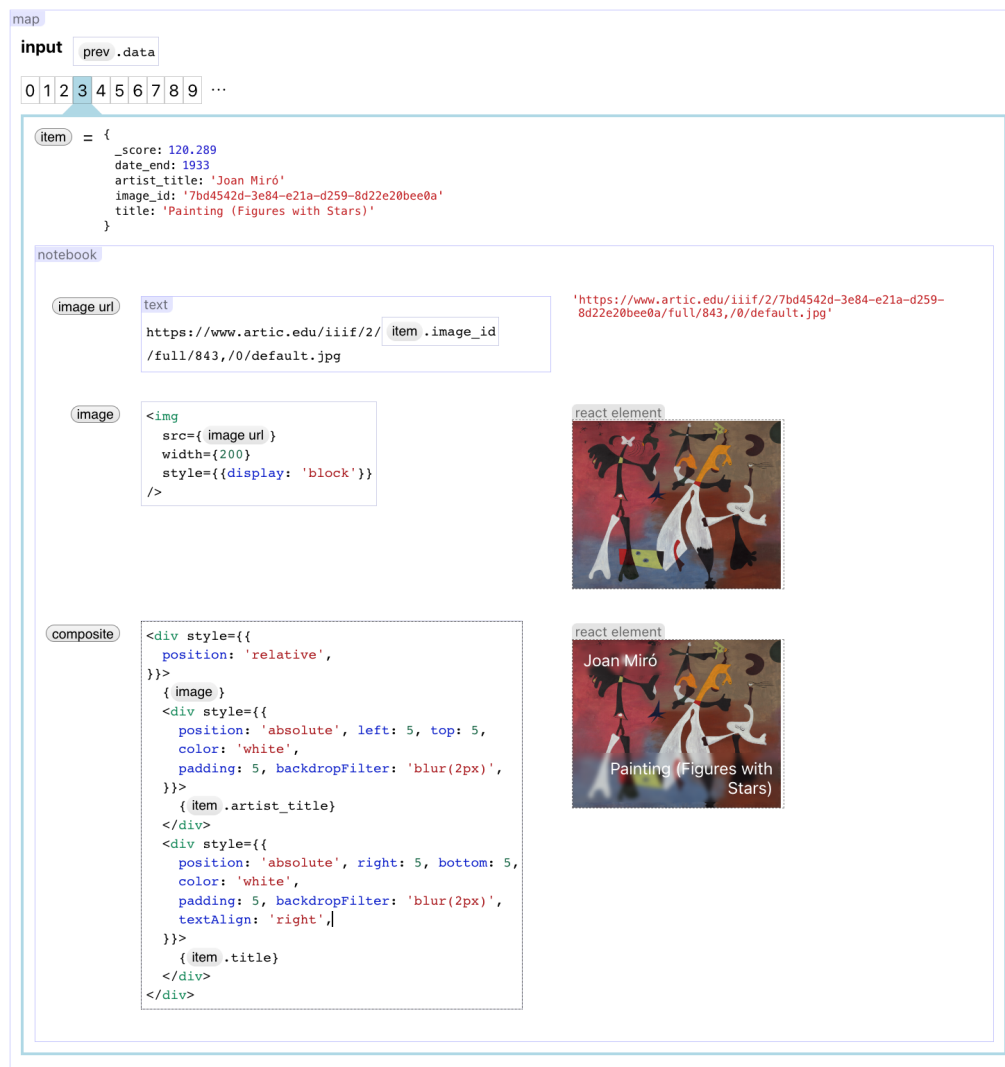


Fig. 12. An excerpt of the program used to generate the image quilt. A Map tool takes in an array of data returned by the Art Institute of Chicago API. A Notebook embedded inside the Map processes each element of this array. In three cells, it 1. constructs an image URL, 2. loads this URL into an image element, to check that it is constructed correctly, and 3. builds a composite, layering text on top of the image element with appropriate styling.

There is nothing special about the example situations described above. Nested structures are pervasive in programming. To provide the benefits of liveness & domain-specific richness across these nested structures, live environments must be similarly composable.

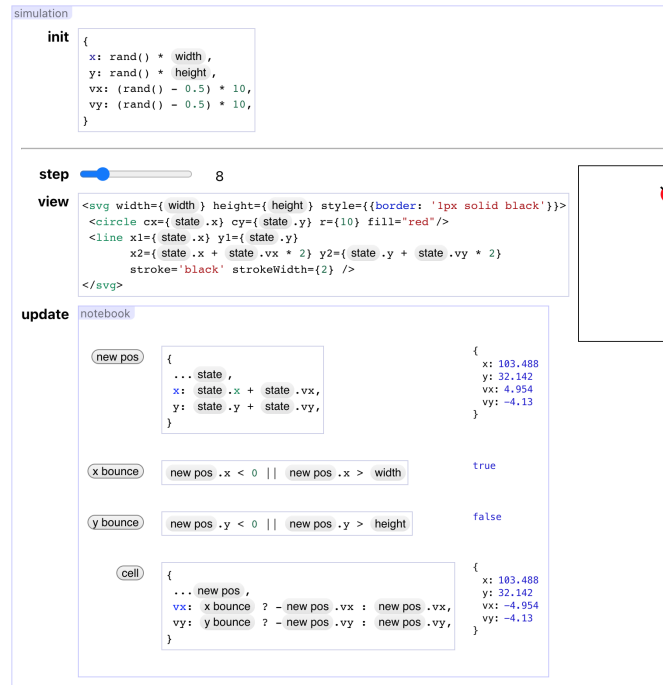


Fig. 13. A Simulation tool, loaded with code that describes how a ball bounces around a rectangular region. The user of the tool has provided 1. init: a slot to initialize the ball's state, 2. view: a slot describing how the state should be rendered, and 3. update: a slot (Notebook, here) describing how the state should be updated on each tick of time. The user has scrubbed the step slider to step 8, at which point the ball is bouncing off the rectangle's right-hand side.

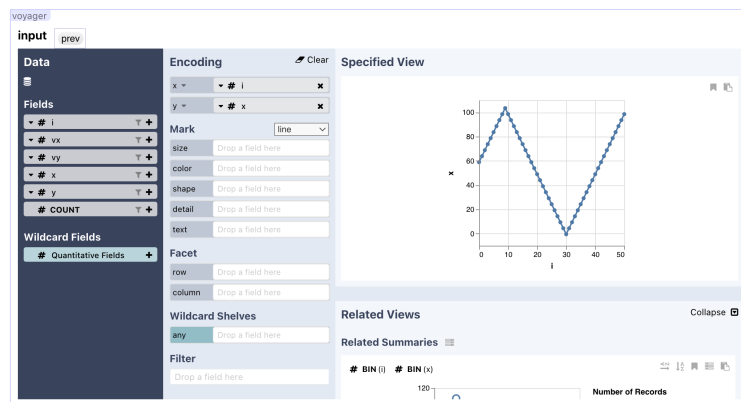


Fig. 14. A Voyager tool in the same Notebook as the Simulation. It has been provided with the Simulation's output as its input, and the user has dragged fields onto the encoding shelves to plot "x" against "i".

3.3 Live tools inside the outside world

Given an Engraff slot, a programmer has access to the entire ecosystem of interoperable Engraff tools. But the question remains of how they get to that slot, and how the program in the slot gets things done in the larger world. Here, we discuss how the Engraff architecture makes it possible to embed live tools and environments where work is done in the real world. We focus on two categories of embeddings: codebases and applications.

3.3.1 Codebases. So far, we have presented Engraff in the context of Graft Garden, a simple web application that hosts Engraff tools and lets users create custom web applications. While Graft Garden is easy to access and use, it naturally has a limited range of usefulness. We do not expect developers of complex web applications to abandon their preferred frameworks, throw out their codebases, and switch to Graft Garden (or any other imagined Engraff host, for that matter). However, we still believe that programmers working in codebases could benefit from the judicious use of live tools and environments, if this didn't require switching entirely into a new, all-encompassing platform.

Fortunately, we have found that the structure of the Engraff ecosystem offers opportunities for integration with present programming practices. With these integrations, programmers can take advantage of what Engraff has to offer in an unobtrusive and gradual fashion.

As an example of this, we prototyped embedding of Engraff tools into conventional web-application programming workflows. Specifically, we implemented a React “hook” called `useLiveTool` which allows a live tool to be embedded into a React codebase. At development time, this hook presents the Engraff user interface running alongside a live version of the web application being developed (Figure 15). Data is fed, live, from the web application being developed into the Engraff user interface. The results are fed, live, back to the web application. When the developer is done working with the tool, they can disable it from being displayed. In production, the “computational behavior” of the tool is used without any visual presentation.

```
const synonyms = useLiveTool(  
  { response },  
  { defaultValue: [], hide: false}  
);
```

This is only one example of how Engraff could be embedded into existing development contexts. Different situations will call for different embeddings. For instance, someone writing a command-line program may want to write a particular function with Engraff. Because the command-line program runs imperatively, with side effects, it can not use `useLiveTool`'s fully-reactive approach where the program re-runs as the user edits the function in Engraff. However, a “programming with examples” approach [26] could be employed, where the user gathers a number of input values for their function before iterating on their function's implementation, testing it on examples as they go.

The design and implementation of these various embeddings will be nontrivial, as they must bridge gaps between a variety of programming paradigms and Engraff's own reactive infrastructure. We discuss some of these challenges briefly in §6.2. However, Engraff's loosely-coupled functional architecture puts it in a good position to take on these challenges.

3.3.2 Applications. Codebases aside, we are also interested in ways interactive end-user applications can host Engraff tools. Graft Garden provides one example, but any application with access to the Web platform could provide an Engraff slot as an access point to the Engraff ecosystem.

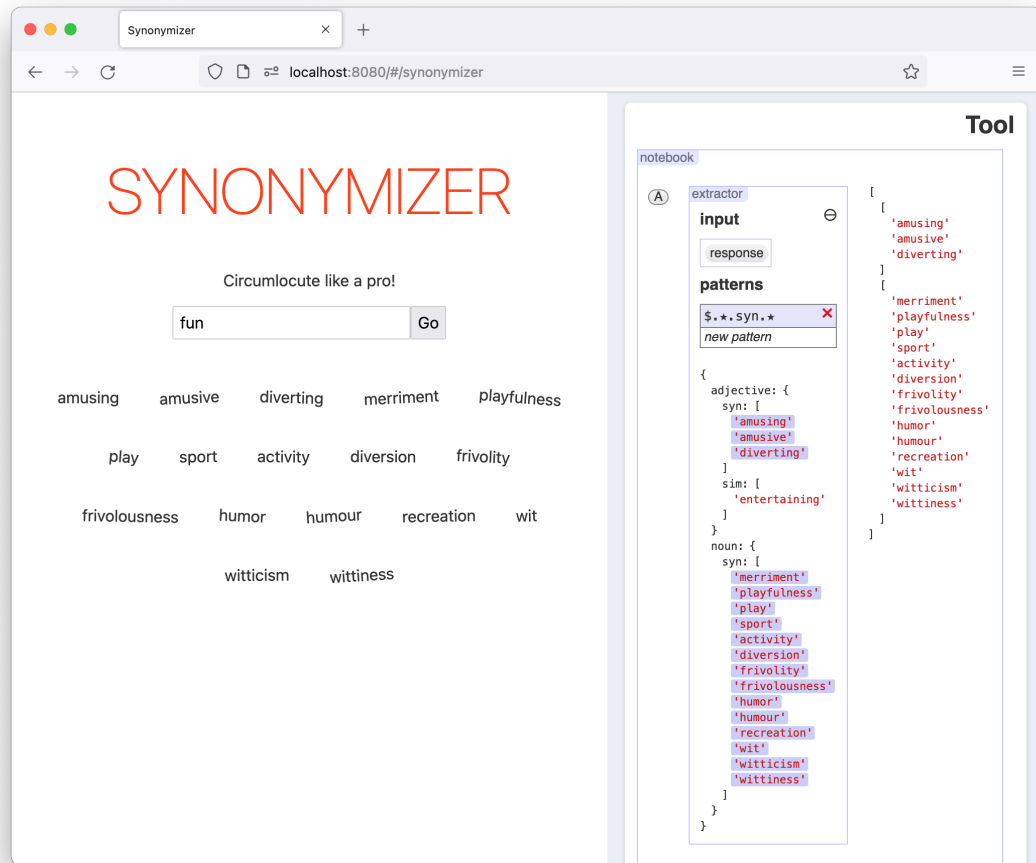


Fig. 15. A conventional web application, Synonymizer, developed with useLiveTool. Because useLiveTool is called with `hide: false`, the live tool is displayed in the browser, to the right of the running application. Several live tools, including Notebook and Extractor, are used in this side pane to transform an API response passed from the conventional code into the tool into a clean set of words that can be displayed on screen. The running app shows the live output from the tool.

For instance, Cuttle [5], a vector editor for digital fabrication, currently has the ability to implement components and modifiers with bits of JavaScript code. If Cuttle's code box were replaced with an Engraft slot, the world of Engraft programming tools would be available, in-place, to Cuttle users. (Figure 16)

One can imagine applications farther from the world of programming, using Engraft to provide open-ended extensions of their own interfaces. An animation application's easing-function editor and color picker could be implemented as Engraft slots prepared with default tools. Experienced users could then choose to remove these defaults and replace them with their own tools. In this way, Engraft could enable end-user customization, blurring the lines between application user and developer.

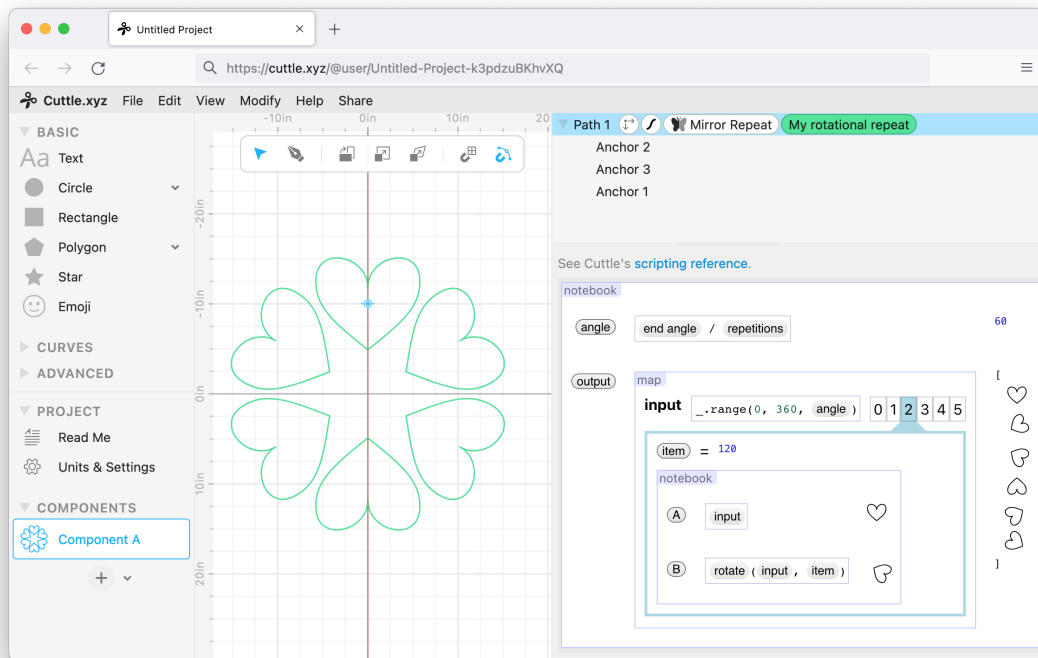


Fig. 16. A mockup of an imagined embedding of Engraff into Cuttle. The Engraff program defines a modifier in Cuttle which transforms an input heart shape into an array of rotated hearts.

4 ENGRAFF DESIGN DECISIONS

The Engraff prototype was developed in accordance with five high-level design decisions.

To support open-ended experimentation in tools, **Engraff is defined as an open API, not a toolkit**. Engraff tools can be implemented in any way, within the bounds of the Engraff API and the Web platform, adopting their own styling, interaction techniques, and implementation frameworks. A toolkit approach would prematurely lock in constraints on tools' designs. An API approach also makes it possible to adapt existing tools to Engraff with minimal changes. As a test of this, we wrapped Voyager 2 [48], a system for visual data exploration, as an Engraff tool. §3.2 shows how this opens up new use-cases for Voyager.

To connect with existing programming activity, and to enable new forms of composition, **Engraff is built on the Web platform**. A large part of the computing world now exists on the Web platform. Engraff can be composed together with this existing activity, as we described in §3.3. In this way, Engraff tools can perform their own specialized roles in a larger ecosystem. This supports gradual adoption of Engraff, creating a feedback loop of development and use. Engraff's use of the Web platform is also the basis for Engraff tools' recursive composability: Engraff tools can host one another because they are both implemented on the Web platform and hosted by the web platform. Systems like mage [19], which do not support open-ended embedding of tools on the Web, lack this.

To support visibility and open-ended composability, **the Engraft API is based on streams of pure values**. A tool's host provides it with a set of variables bound to JavaScript values. The tool can, at any point, provide output (a JavaScript value or error) through a host-provided callback. We chose to base communication on a functional, value-oriented paradigm due to the success this paradigm has had in diverse contexts ranging from React to spreadsheets.

To support both live editing and serious computational use, **Engraft makes tool-running primary and tool-rendering secondary**. The Engraft API lets a tool run, taking in input from its host and returning output, without rendering any UI. As part of running, the tool offers its host an optional UI. In many situations, the host will use the option of *not* displaying a tool's UI, such as when a codebase embeds a live tool in production or when a tool like Map only renders an example instance of its body. By allowing tools to run invisibly, Engraft supports these serious computational uses. This relationship between running and displaying is not true the other way around: an Engraft tool's UI *cannot* be displayed without running it. Requiring that tool developers create versions of their tools which can run in static environments would be a distraction from Engraft's goal of supporting pervasively live programming.

To encourage innovative interactive tools, **Engraft tools are not required to generate or edit underlying textual code**. In Engraft, a tool's underlying state is not code, but a serializable JavaScript object we call its *program*. This program can take any form, and can be interpreted in any way at runtime by the tool's implementation. This stands in contrast to a common approach taken by live-programming systems, in which a live tool ultimately generates source code which defines its computational behavior, and is often expected to re-parse this code after it has been manually edited. While it is true in a technical sense that any serializable JavaScript object could be represented as textual code and visa versa, the two formats have different grains. In platforms based on tools generating textual code, tool designers are encouraged to make tools based on pre-existing code patterns. With Engraft, we want tool designers to start with a task, and freely determine their design without regard to textual-code idioms.

5 ENGRAFT IMPLEMENTATION DETAILS

We now present implementation details of the Engraft prototype. We begin with a slightly idealized description of the Engraft API, before explaining a few deviations from this ideal, then go on to discuss the Engraft tool ecosystem.

5.1 The Engraft API

An Engraft tool is implemented as a JavaScript class that abides by an interface that Engraft specifies. A host uses this interface to create and manage an instance of this tool it wants to embed. Since all tools abide by the same interface, a host that uses this interface can embed any tool.

A tool class's interface is centered around a particular bundle of “properties” that contains all the information a host provides to an embedded tool. When a host creates a tool instance, it provides it with a set of properties (1, below). The host can later call an “update” method on a tool instance to provide a new set of properties (2, below). Finally, the host can call a “destroy” method when the tool is no longer needed (3, below).

```
const toolInstance = new MyTool(properties); // 1
toolInstance.update(newProperties);           // 2
toolInstance.destroy();                       // 3
```

The real content of the Engraft API lies in the fields of the properties object. They are as follows:

- **program**: The tool's program, a serializable JavaScript object that defines its behavior.
- **updateProgram**: A callback the tool uses to request a change to its program.

- `reportOutput`: A callback the tool uses to report its output – a value or error.
- `reportView`: A callback the tool uses to report its view (UI).
- `varBindings`: Bindings of variable ids & names to actual values, which the tool can immediately access.

A tool with access to the above five values has everything it needs to operate. It can read in its program specification, grab bindings from the environment (if needed), perform computations, and report back output and a view. A reported view can display any information gathered during the tool’s execution, including intermediate values and final output. This view can also contain interactive controls which, when acted upon by the user, will trigger `updateProgram` to modify the tool’s program.

This design was inspired by the component model of React [24], a library for building user interfaces. React components offer a similar interface, in which a bundle of “props” is provided by an owner when a component is created, and then updated whenever the owner re-renders. React offers a set of tools for efficiently managing recomputation when a component’s props change, including “hooks” and higher-order components for memoization.

React’s model fit Engraff’s needs well enough that we chose to implement our prototype system directly using React. This means that an Engraff tool is defined as a React component, rather than as the class we described above. Our use of React here is quite idiosyncratic. React components usually return trees of HTML elements, but in Engraff, it is essential that view behavior remain optional, and secondary to computational behavior. So Engraff tool components never directly return any HTML elements. When React “renders” a tool, it will construct an abstract tree that stores and maintains the state of the tool, together with the state of any subtools it may contain. These tools can then use callbacks provided as props to report output values and UI views back up the tree.

In our use of React, the first four “properties” above are provided directly to tools as React “props”. The last property, `varBindings` is provided indirectly, using React’s “context” feature. By wrapping a React component in context, the entire subtree under this context gets access to this information. Since most tools pass down the bindings they receive from their parent unchanged to their children, context provides a better developer experience here than props.

In addition to providing a React component (in lieu of the class we first described), a tool also provides a function called `programFactory` which is used to initialize a program for a new instance of the tool. To support the common pattern of a tool having a “main input” which an environment may want to pre-populate, `programFactory` optionally accepts a string `defaultInputCode`. This allows ergonomic interactions, such as new cells in a notebook automatically taking in the previous cell as input.

For more details, the supplemental material includes example code for several small tools. React has served as a helpful infrastructure for Engraff; nevertheless, it may be worth reconsidering in future work, as it is a heavyweight, unorthodox dependency.

5.2 Slots and Subtools

Throughout this paper, we have shown Engraff programmers using “slots” to compose nested programs. The Slot tool is the glue that holds together Engraff programs. It is a built-in tool that appears at first as a code editor. Arbitrary JavaScript can be entered into this code editor, where it is compiled, evaluated, and returned as output. References to Engraff variables can be inserted into this editor using an auto-complete window. If a tool’s name is selected via auto-complete, the slot will be replaced with the tool, entering “subtool mode” (Figure 17).

When Slot renders a subtool, it provides it with a “tool frame” to identify it. On hover, the title bar of this frame also reveals a few buttons (Figure 18):

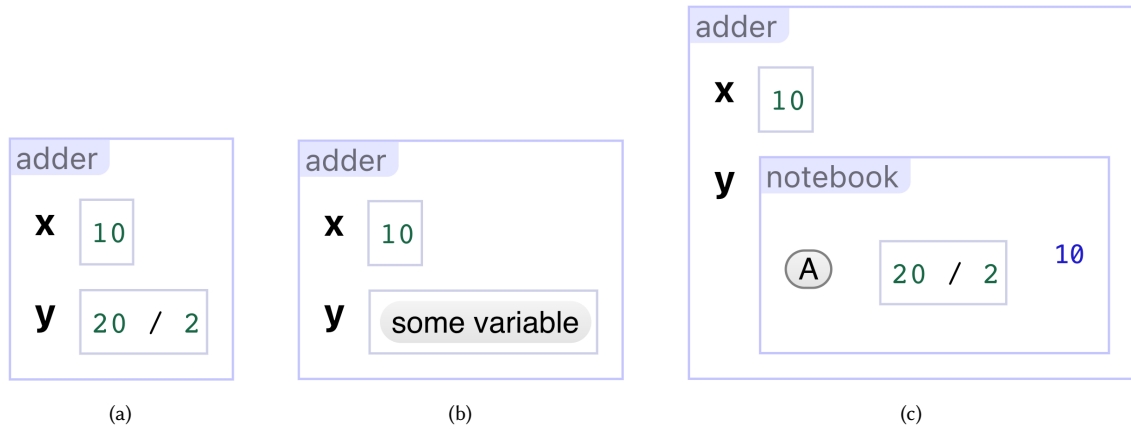


Fig. 17. A simple Adder tool. Its “y” slot is provided with (a) a code snippet, (b) a reference to a variable provided by the Adder’s host, (c) a nested Notebook.

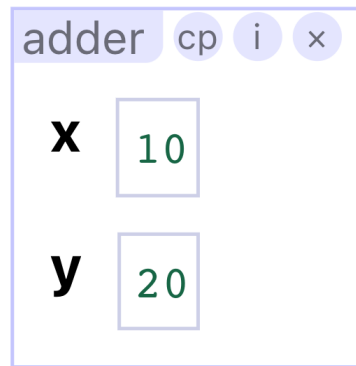


Fig. 18. The appearance of an Adder tool’s “tool frame” when its title bar is hovered, revealing three circular buttons.

- cp: Copy the tool’s program to the clipboard, so it can be pasted into a different location.
- i: Display a pop-up debugger window with information on the tool’s program and environment.
- x: Remove the subtool, bring the slot back into code mode.

We anticipate this frame will support additional general-purpose interactions in the future, such as “maximizing” a tool for focused work or “pinning” it to a sidebar.

Through the use of slots, a “tree of tools” is formed during the use of Engraft. Figure 19 shows an example arrangement of tools together with a diagram of the resulting tree. Note, however, that there is no explicit reference to a tree of tools anywhere in the Engraft API or implementation. Rather, this tree emerges from the fact that a tool, like any other software system, is free to act as a host that embeds tools.

Although it is not built into the Engraft API (which could theoretically be used without it), Slot plays an essential role in the Engraft user experience. While the Engraft API provides the computational structures needed for open-ended composition, Slot provides users with an interface that makes that composition accessible in practice.

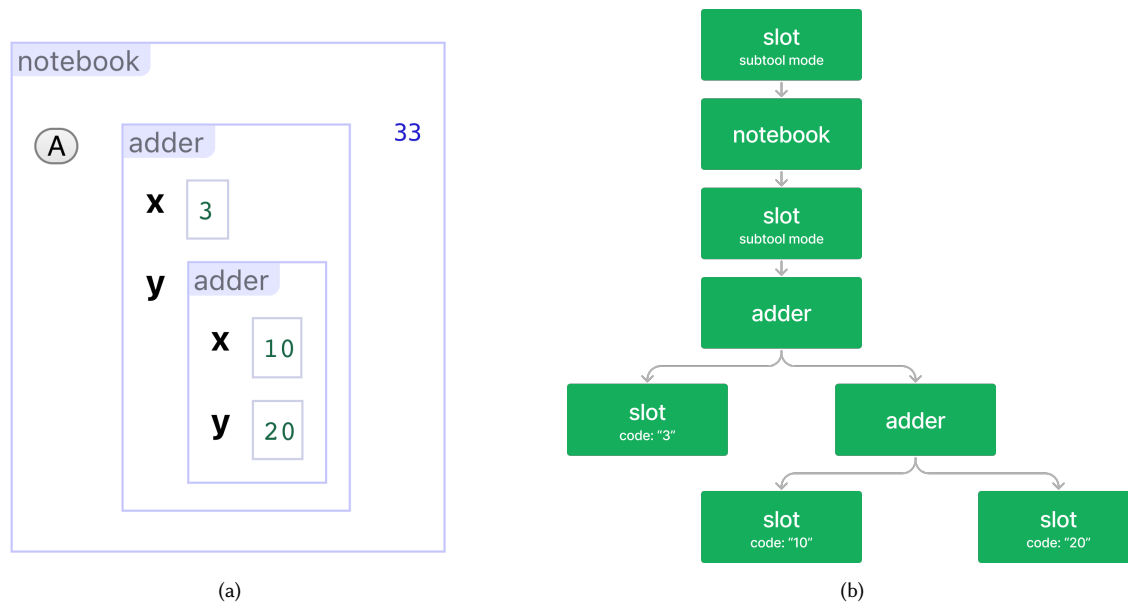


Fig. 19. (a) An arrangement of an Adder nested in an Adder nested in a Notebook. (b) The tree of tools resulting from this arrangement. Note the Slot tools, which provide the code-editors shown as leaves of the tree, as well as invisible intermediates between tools and their subtools.

5.3 Where do tools come from?

We showed earlier how a tool can be implemented as a JavaScript module. This technical description raises a few questions:

How does a prospective tool user access one of these modules? In our current prototype, we side-step this question – all tools are bundled together with the Engraff system. For actual use, it will be important for tool creators to be able to post tools publicly and for tool users to be able to access them easily. One convenient approach would be to leverage existing package managers like NPM.

Does every tool need to be coded from scratch, by a developer intimately familiar with Engraff's low-level API? Right now, yes. But we see a range of possibilities that would support more accessible tool creation. For instance:

- A user could build a composite of Engraff tools (e.g., a color picker built in a computational notebook) and then select parts of this composite to expose in a custom tool interface.
- A user could take an existing web-based tool, and then describe how it could be embedded into an i frame and driven to make it participate in the Engraff API.

At the same time, our preferred design is to build these higher level meta-tools on top of the minimal Engraff API, which we expect is general enough to support them.

5.4 Tools built so far

To date, we have implemented a few dozen primitive tools with Engraff. The examples in §3 highlight some of these: Notebook, Request, Extractor, Formatter, NotebookCanvas, Map, Simulation, and Voyager. These examples also include

Slider, Color, and Text tools. Appendix A includes pictures and brief descriptions of eight more tools. These tools are all “sketches”, intended to test the Engraft API’s design and to demonstrate its combinatorial possibilities. We expect that, with continued use, we will see both more polished and more radically divergent tools enter the Engraft ecosystem.

6 DISCUSSION

Our starting point in this paper is a simple observation: In spite of enormous efforts from industry and academia spanning decades, live-programming tools are not a mainstream part of programming practice. To our knowledge, this fact has not found much discussion in the literature.¹⁰ We believe that, given the enormous potential live programming might offer, this gap should be confronted head-on.

This paper proposes *composability* as an essential element which must be combined with *liveness* and *domain-specific richness* to make alternatives to textual code effective in practice. We have presented Engraft: a prototype API which combines these three elements, enabling new forms of composition.

While we expect the lack of adoption of live tools is due to many factors, and there is no silver bullet that will make live tools truly usable, we believe that the approach Engraft makes concrete is compelling and worth further exploration.

6.1 Risks

There are numerous ways Engraft may fail to work in practice.

Engraft is heterogeneous. We are excited by the prospect of Engraft fostering a vibrant, diverse, anarchic ecosystem of tools and environments. But our embrace of heterogeneous implementations carries liabilities. It might be challenging for users to learn new tools. It might be uncomfortable to work in a space that ties together discordant interfaces. Interoperability between tools might be a problem, since data exchange formats are not enforced.

Engraft is untyped. Engraft is built off the Web platform, and embraces JavaScript’s highly dynamic, untyped style. This stands in contrast to the closely aligned livelits project [29], discussed earlier, which specifically builds on Hazel’s strong type discipline. Engraft may fail to scale to more complex work, if it turns out the benefits of static types are essential for managing this complexity. (We suspect that, in many cases, pervasive liveness forms an alternative to static types, but this is a speculative hypothesis.)

Engraft is non-textual. Engraft is, ultimately, a structured editor, as an Engraft program is not manipulated entirely as text. This is a fraught path. Plain-text interactions are familiar and well-polished by decades of computing. Structured editors can easily fail to match their fluidity and ergonomics, resulting in frustration from users. Textual code is also assumed by many important workflows, such as version control, and it is not immediately clear how these can be extended to non-textual programs. Recent work has sought to address some of these issues (such as Moon & Omar [25]’s work on token-level editing and Edwards & Petricek [8]’s work on version control), but this is still an active area of research.

Engraft tools are mostly isolated from one another. An Engraft “tool” is analogous to a function or other bit of syntax in a functional programming language. It can access bindings provided by its host, return a value back to the host, and that’s it. This design follows a traditional functional-programming discipline, which brings many advantages, but which also constrains the space of what a tool can be. An illustrative example comes from Gneiss [4]. Gneiss

¹⁰Tanimoto’s 2013 reflections on live systems [43] includes a section on “Criticisms of Liveness”, though he quickly dismisses them in favor of a generally optimistic view. Lau’s short piece [20] on why programming-by-demonstration systems (a related category) sometimes fail is insightful, but it focuses on AI-specific aspects and does not apply to live-programming systems in general.

combines three separate tools: an API query tool, a spreadsheet, and an interface builder. We wondered how Gneiss could be “unbundled” with Engraft, so that the tools that make it up could be used in new contexts and different tools could be brought into Gneiss’s context. We found that, while it is possible to construct a workflow similar to Gneiss in Engraft today, the experience does not perfectly replicate Gneiss. Gneiss uses interactions that tightly tie its different tools together, like dragging a JSON field from the API query tool directly to the spreadsheet. It also uses flows of data which are, from a functional-programming perspective, quite messy, such as cyclic data flow between the spreadsheet and the interface builder. These are not presently possible with Engraft. It remains to be seen whether a programming environment like Engraft, based on isolated functional tools, is adequate to produce a holistically rich programming experience, or what would need to be added to the Engraft model to make it so.

6.2 Technical Limitations

Aside from those large unknowns, we already anticipate some ways Engraft will need to be developed to support realistic use.

Engraft’s **use of the Web platform** is a source of much of its potential, but it also brings along tricky issues. Security is naturally a concern. Web-based programming platforms (like Observable and Natto) have had to put much effort into sandboxing, and we can expect Engraft’s flexibly-nested structures would make this all the more difficult. Another source of challenges is layout and styling. Engraft tools push the boundaries of UI frameworks, since tools act as heterogeneous composable application interfaces. CSS and the rest of the Web platform were not designed to support anything like this. They have worked well for prototyping so far, but careful work is required to find approaches that ensure the system stays robust, even as diverse tools are added.

Engraft’s current **reactivity model** is limited. Currently, communication between tools and hosts is a free-for-all. A host can deliver new values to a tool whenever it likes, and the tool can reply with output whenever *it* likes. But there are situations where a greater degree of control is desirable. Certain hosts may want to run a tool synchronously, or run it asynchronously but in such a way that it can know for sure when the tool’s output is “up to date” with its inputs. Neither of these is possible right now. Changes to the reactivity model could also aid performance, as right now “glitches” can cause unnecessary computations on out-of-date inputs.

Engraft’s **embedding into conventional codebases** is incomplete, and there are details in the workflow that need working out. Many uses of live tools from conventional code will need forms of reactivity that are not currently supported, as described in the last paragraph. There are also questions around how the “program” of a live tool should be persisted. Currently, our prototype of `useLiveTool` simulates persistence by storing a tool’s program in the user’s web browser local storage. But for any real use, it will be important for this program to be stored as part of the web application’s codebase, where it can be delivered with the application for production use, checked into source control, etc. This means live tools will need to write back to the codebase during development. Careful engineering and design work is required here, as live tools are competing against refined and entrenched textual workflows.

6.3 Future Work

While Engraft has been prototyped well enough to allow experimental use, it has not reached a stage where research would benefit from broad promulgation and use in practice. This is especially true because Engraft takes the form of an API standard. If tools and hosts were built around one version of this API and then future discoveries suggested moving to a new version, effort would be wasted, incompatibilities would proliferate, and confidence would be undermined. Or,

perhaps worse, dependencies on one version of Engraft's design might discourage iteration and exploration of Engraft itself, prematurely locking in an underdeveloped design.

Given this, there are several appealing ways to continue developing and testing Engraft. One is to dive into a particular domain where end-user programming feels like a bottleneck, partnering closely with practitioners to test and iterate on the design of Engraft as it applies to their work. Once Engraft's design can be validated through successful application in multiple domains (say, both creative coding and data science), it may be ready to grow more openly.

Another is to establish Engraft as an enabling platform for researchers developing experimental live tools. Suppose a researcher has an idea for a novel live tool. Without Engraft, they are in a tough spot. As effective as their tool may be for its task, evaluating the tool requires building enough complementary infrastructure that their novel tool can be demonstrated in an end-to-end workflow. This may involve a significant amount of work which is unrelated to the novel interactions they are exploring. If the researcher implements their tool on top of Engraft, however, they can test it in the context of all the tools, environments, and real-world embeddings that already exist in the Engraft ecosystem. We hope that, in this way, Engraft can lower the barrier to entry for live-tool research and accelerate innovation in this field.

REFERENCES

- [1] Leif Andersen, Michael Ballantyne, and Matthias Felleisen. 2020. Adding interactive visual syntax to textual code. *Proceedings of the ACM on Programming Languages* 4 (nov 13 2020), 1–28.
- [2] Apple Inc. 2014. WWDC 2014. <https://www.youtube.com/watch?v=w87fOAG8fjk>.
- [3] Bootstrap. 2022. Bootstrap. <https://bootstrapworld.org/index.shtml>.
- [4] Kerry Shih-Ping Chang. 2016. A Spreadsheet Model for Using Web Services and Creating Data-Driven Applications. *Carnegie Mellon University* (2016).
- [5] Cuttle Labs Inc. 2022. Cuttle - Design tool for digital cutting machines. <https://cuttle.xyz/>.
- [6] Jonathan Edwards. 2004. Example centric programming. *ACM SIGPLAN Notices* 39, 12 (12 2004), 84–91.
- [7] Jonathan Edwards. 2005. Subtext. In *Proceedings of the 20th annual ACM SIGPLAN conference on Object oriented programming systems languages and applications - OOPSLA '05*. ACM Press.
- [8] Jonathan Edwards and Tomas Petricek. 2021. *Typed Image-based Programming with Structure Editing*. arXiv.
- [9] T. Ellis, J. Heafner, and W. Sibley. 1969. *The GRAIL Language and Operations*. RAND Corporation.
- [10] T. O. Ellis, John F. Heafner, and William L. Sibley. 1969. THE GRAIL PROJECT: AN EXPERIMENT IN MAN-MACHINE COMMUNICATIONS. https://www.rand.org/pubs/research_memoranda/RM5999.html.
- [11] Enso. 2022. Enso. <https://enso.org/>.
- [12] Kasra Ferdowsifard, Allen Ordookhanians, Hila Peleg, Sorin Lerner, and Nadia Polikarpova. 2020. Small-Step Live Programming by Example. In *Proceedings of the 33rd Annual ACM Symposium on User Interface Software and Technology*. ACM.
- [13] Chris Granger. 2022. Light Table. <http://lighttable.com/>.
- [14] Joshua Horowitz. 2018. PANE: Programming with Visible Data. (2018). Presented at the Workshop on Live Programming (LIVE) 2018.
- [15] Tomoki Imai, Hidehiko Masuhara, and Tomoyuki Aotani. 2015. Shiranui: a live programming with support for unit testing. In *Companion Proceedings of the 2015 ACM SIGPLAN International Conference on Systems, Programming, Languages and Applications: Software for Humanity*. ACM.
- [16] JetBrains s.r.o. 2022. MPS: The Domain-Specific Language Creator by JetBrains. <https://www.jetbrains.com/mps/>.
- [17] Sean Kandel, Andreas Paepcke, Joseph Hellerstein, and Jeffrey Heer. 2011. Wrangler. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. ACM.
- [18] Saketh Kasibatla and Alex Warth. 2017. Seymour: Live Programming for the Classroom. (2017). Presented at the Workshop on Live Programming (LIVE) 2017.
- [19] Mary Beth Kery, Donghao Ren, Fred Hohman, Dominik Moritz, Kanit Wongsuphasawat, and Kayur Patel. 2020. mage: Fluid Moves Between Code and Graphical Work in Computational Notebooks. In *Proceedings of the 33rd Annual ACM Symposium on User Interface Software and Technology*. ACM.
- [20] Tessa Lau. 2009. Why Programming-By-Demonstration Systems Fail: Lessons Learned for Usable AI. <https://doi.org/10.1609/aimag.v30i4.2262>. *AI Mag.* 30, 4 (2009), 65–67.
- [21] Sorin Lerner. 2020. Projection Boxes: On-the-fly Reconfigurable Visualization for Live Programming. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. ACM.

- [22] Zhicheng Liu, John Thompson, Alan Wilson, Mira Dontcheva, James Delorey, Sam Grigg, Bernard Kerr, and John Stasko. 2018. Data Illustrator. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*. ACM.
- [23] Eyal Lotem and Yair Chuchem. 2022. Lamdu. <https://www.lamdu.org/>.
- [24] Meta Platforms, Inc. 2022. React – A JavaScript library for building user interfaces. <https://reactjs.org/>.
- [25] David Moon and Cyrus Omar. 2021. Restructuring Structure Editing. (2021). Presented at the Workshop on Live Programming (LIVE) 2021.
- [26] B. A. Myers. 1986. Visual programming, programming by example, and program visualization: a taxonomy. In *Proceedings of the SIGCHI conference on Human factors in computing systems - CHI '86*. ACM Press.
- [27] Observable Inc. 2022. Observable - Explore, analyze, and explain data. As a team. <https://observablehq.com/>.
- [28] Observable Inc. 2022. Quickly Explore & Analyze Your Data With Data Table Cell / Observable / Observable. <https://observablehq.com/@observablehq/introducing-data-table-cell>.
- [29] Cyrus Omar, David Moon, Andrew Blinn, Ian Voysey, Nick Collins, and Ravi Chugh. 2021. Filling typed holes with live GUIs. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. ACM.
- [30] Cyrus Omar, Ian Voysey, Ravi Chugh, and Matthew A. Hammer. 2019. Live functional programming with typed holes. *Proceedings of the ACM on Programming Languages* 3 (jan 2 2019), 1–32.
- [31] C. Omar, YoungSeok Yoon, T. D. LaToza, and B. A. Myers. 2011. Active code completion. In *2011 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. IEEE.
- [32] Everest Pipkin. 2021. been having some motivation troubles recently (god who hasn't) so i'm gonna pick a tiny personal project off my ideas list and see if i can get it working by morning, tonight - a lil bash script that emails me the summaries of 5 random wikipedia articles each morning. <https://twitter.com/everestpipkin/status/1349274983651012609>.
- [33] Joe Politz, Benjamin Lerner, Sorawee Porncharoenwase, and Shriram Krishnamurthi. 2019. Event Loops as First-Class Values: A Case Study in Pedagogic Language Design. *The Art, Science, and Engineering of Programming* 3, 3 (feb 1 2019).
- [34] David Rauch, Patrick Rein, Stefan Ramson, Jens Lincke, and Robert Hirschfeld. 2019. Babylonian-style Programming: Design and Implementation of an Integration of Live Examples into General-purpose Source Code. *The Art, Science, and Engineering of Programming* 3, 3 (feb 1 2019).
- [35] Patrick Rein, Stefan Ramson, Jens Lincke, Robert Hirschfeld, and Tobias Pape. 2018. Exploratory and Live, Programming and Coding. *The Art, Science, and Engineering of Programming* 3, 1 (jul 23 2018).
- [36] Donghao Ren, Bongshin Lee, and Matthew Brehmer. 2019. Charticulator: Interactive Construction of Bespoke Chart Layouts. *IEEE Transactions on Visualization and Computer Graphics* 25, 1 (1 2019), 789–799.
- [37] Arvind Satyanarayan and Jeffrey Heer. 2014. Lyra: An Interactive Visualization Design Environment. *Computer Graphics Forum* 33, 3 (6 2014), 351–360.
- [38] Yoshiki Schmitz. 2019. I've been jamming on this concept for making data-driven designs... <https://twitter.com/yoshikischmitz/status/1176642448077967362>.
- [39] Paul Shen. 2021. Show HN: Natto – a canvas for writing and manipulating JavaScript. <https://news.ycombinator.com/item?id=26478548>.
- [40] Paul Shen. 2022. welcome! – natto. <https://natto.dev/>.
- [41] Frank M. Shipman and Catherine C. Marshall. 1999. Formality Considered Harmful: Experiences, Emerging Themes, and Directions on the Use of Formal Representations in Interactive Systems. *Computer Supported Cooperative Work (CSCW)* 8, 4 (12 1999), 333–352.
- [42] Steven L. Tanimoto. 1990. VIVA: A visual language for image processing. *Journal of Visual Languages & Computing* 1, 2 (6 1990), 127–139.
- [43] Steven L. Tanimoto. 2013. A Perspective on the Evolution of Live Programming. In *Proceedings of the 1st International Workshop on Live Programming (LIVE '13)*. IEEE Press, San Francisco, California, 31–34.
- [44] Tijs van der Storm and Felienne Hermans. 2016. Live Literals. (2016). Presented at the Workshop on Live Programming (LIVE) 2016.
- [45] Bret Victor. 2012. Inventing on Principle. (2012). Presented at the the Canadian University Software Engineering Conference (CUSEC).
- [46] Bret Victor. 2012. Learnable Programming. <http://worrydream.com/LearnableProgramming/>.
- [47] Wikipedia contributors. 2022. *Grafting — Wikipedia, The Free Encyclopedia*. [Online; accessed 01-September-2022].
- [48] Kanit Wongsuphasawat, Zening Qu, Dominik Moritz, Riley Chang, Felix Ouk, Anushka Anand, Jock Mackinlay, Bill Howe, and Jeffrey Heer. 2017. Voyager 2. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. ACM.

A TOOL MENAGERIE



Fig. 20. “Import” allows modules to be dynamically imported from NPM.



Fig. 21. “File” lets the user upload a file into the browser by dragging and dropping it onto the tool. The file is then persistently saved in the Engraff program as a data URI string.

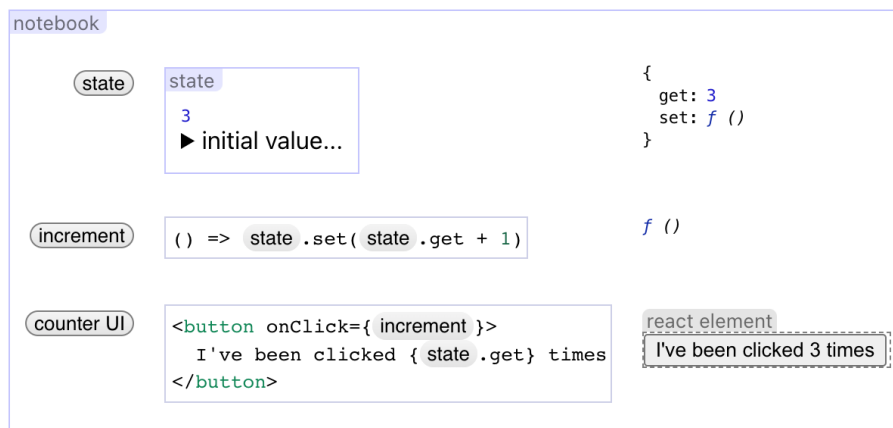


Fig. 22. “State” stores a piece of ephemeral state. It returns the current value of this state together with a setter. This can be used to implement stateful UIs.

```
chalk
input "David Attenborough"

1 function (x) {
2   const parts = x.split(' '); ['David', 'Attenborough']
3   const initials = parts.map(w => w[0]); ['D', 'A']
4   return initials.joi(' '); runtime: initials.joi is not a function
5 }
```

Fig. 23. “Chalk” is a code editor that displays live values and errors inline.

```
synthesizer
input "Input Without Provided Output"

"George Washington Carver" → "GWC" ✗
"David Attenborough" → "DA" ✗
"Input Without Provided Output" → "IWPO"

Run

code input.split(' ').map((x) => x[0]).join('')
```

Fig. 24. “Synthesizer” is a simple example-based program synthesizer, following the example of [12].

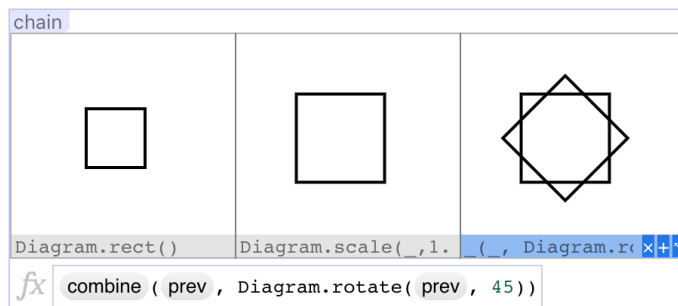


Fig. 25. “Chain” is a minimalist live environment inspired by the data-first design of spreadsheets.

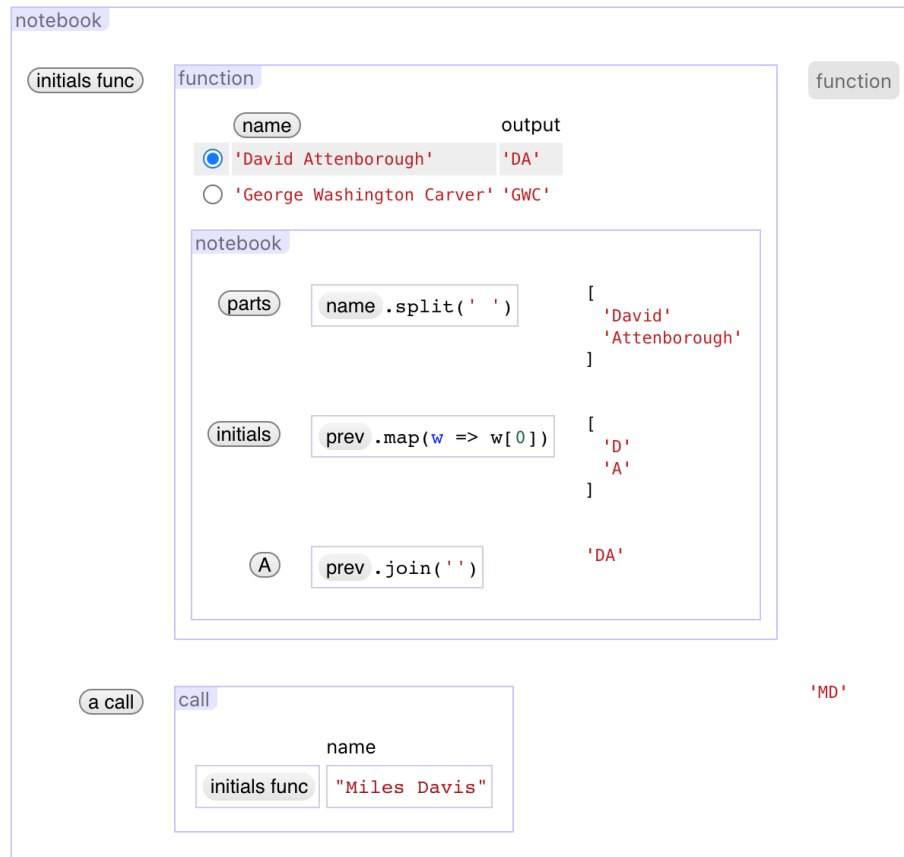


Fig. 26. “Function” can be used to define a function, using examples for live feedback. “Call” can then be used to call the function.